

A Multi-Agent LLM Architecture for Natural Language Access to Relational Business Data

Tien Thanh Pham¹ and Tuan-Dat Trinh¹

School of Information and Communications Technology, Hanoi, Vietnam
tienthanhstk11@icloud.com, dattt@soict.hust.edu.vn

Abstract. Natural Language to SQL (NL2SQL) reduces the barrier between business users and relational data, yet large language model (LLM)-based systems still struggle with schema grounding, join selection, aggregation, and nested logic. This paper presents an application-oriented study of a six-stage multi-agent NL2SQL pipeline with explicit reasoning checkpoints: Question Analysis, Schema Selection, Query Planning, SQL Generation, SQL Refinement, and SQL Validation. Rather than treating SQL generation as a single step, the architecture isolates key decision points to improve error control. The system is evaluated under a fixed protocol on the Spider 1.0 development split (1,034 questions), adopted due to the unavailability of the official test server. The proposed pipeline achieves 77.8% Exact Match (EM) and 85.6% Execution Accuracy (EX), outperforming a matched 4-stage baseline (73.7% EM, 81.2% EX). These results indicate that explicit reasoning decomposition improves robustness, particularly for structurally complex queries. However, the gains cannot yet be cleanly attributed to the architecture itself, as they may partly stem from the increased inference budget. The paper therefore frames the contribution as a controlled architectural study rather than a leaderboard claim, and reports detailed configuration settings to support reproducibility.

Keywords: NL2SQL, Text-to-SQL, multi-agent systems, large language models, business analytics, decision support

1 Introduction

Natural Language to SQL (NL2SQL) translates natural language questions into executable SQL queries over relational databases, enabling non-technical users to access structured data without writing code. This capability is central to business analytics and decision-support systems.

Despite recent advances in large language models (LLMs), NL2SQL remains challenging for queries requiring multi-step reasoning. Typical failure modes include incorrect output fields, invalid join paths, missing aggregation constraints, and syntactically correct SQL queries that appear plausible but fail to reflect the actual intent of the question.

A common limitation of existing NL2SQL systems is their reliance on centralized generation. In such designs, schema grounding, join reasoning, aggregation,

and output selection are implicitly entangled within a single generation step. As a result, early-stage reasoning errors are rarely recoverable and often propagate to the final query.

This paper investigates whether explicitly decomposing NL2SQL into intermediate reasoning stages improves robustness in complex query generation. We propose a six-stage pipeline consisting of (1) Question Analysis, (2) Schema Selection, (3) Query Planning, (4) SQL Generation, (5) SQL Refinement, and (6) SQL Validation. Each stage focuses on a distinct reasoning task, producing explicit intermediate outputs that serve as inputs to subsequent stages.

The system is evaluated on Spider 1.0 [16], a widely used cross-domain Text-to-SQL benchmark in which training and evaluation use different database schemas to assess cross-schema generalization. Since official evaluation on the hidden Spider 1.0 test set requires submission to a dedicated evaluation server that is no longer operational, all experiments in this study are conducted on the development split.

The proposed 6-step pipeline achieves 77.8% Exact Match (EM) and 85.6% Execution Accuracy (EX), outperforming a matched 4-step baseline (73.7% EM, 81.2% EX). While these results suggest that explicit reasoning decomposition may improve SQL generation accuracy, the current study does not fully isolate the benefits of decomposition from the additional inference budget required by the multi-stage architecture.

All reported results are obtained with a single fixed configuration of models and hyperparameters (model assignment and parameters in Table 2), selected through internal benchmarking before the final evaluation runs and kept unchanged across all experiments.

This work makes three main contributions. (i) First, it proposes an explicit reasoning-decomposition framework for NL2SQL in which intermediate decisions are externalized rather than remaining implicit, enabling inspection, refinement, and improved error containment throughout the generation process. (ii) Second, it provides a controlled empirical comparison between a reduced 4-stage baseline and the full 6-stage architecture on the Spider development split, enabling a systematic assessment of the impact of reasoning decomposition under a consistent experimental setup. (iii) Third, it presents a detailed analysis of performance across query difficulty levels and recurring error categories, offering insights into where multi-stage decomposition is most beneficial and where limitations remain.

2 Related Work

Early neural Text-to-SQL systems, such as the feedback-driven neural semantic parser of Iyer et al. [4] and SyntaxSQLNet [15], established the task of translating natural language questions into executable SQL queries. The introduction of Spider [16] shifted the focus toward cross-domain generalization, requiring models to reason over previously unseen database schemas. Subsequent systems, including RAT-SQL [12], BRIDGE [6], and RESDSQL [5], substantially

improved schema-aware representation and schema linking, leading to stronger performance on complex multi-table queries.

The emergence of large language models (LLMs) further transformed the NL2SQL landscape. Prompting-based approaches such as DIN-SQL [9] and DAIL-SQL [2] demonstrated that general-purpose LLMs can generate competitive SQL without task-specific decoders. Nevertheless, SQL generation is commonly performed within a single generation stage, making errors in schema grounding, join selection, or aggregation difficult to detect before they propagate to the final query.

To address these limitations, recent studies have increasingly explored explicit reasoning decomposition, iterative refinement, and multi-agent collaboration. SGU-SQL [17] introduces structure-aware decomposition to improve schema linking and SQL construction, while MAC-SQL [13] coordinates selector, decomposer, and refiner agents to improve execution accuracy through multi-agent collaboration. More general reasoning frameworks such as Reflexion [11], CRITIC [3], AutoGen [14], LangChain [1], and CrewAI [7] further demonstrate the benefits of iterative reasoning and coordinated agents for complex language tasks. Building on this direction, DIVER [8] integrates interactive tool use, dynamic value linking, and evidence-based reasoning to improve robustness in more realistic Text-to-SQL scenarios. Complementary approaches, including constrained decoding with PICARD [10], improve syntactic validity through decoder-level constraints rather than reasoning decomposition.

Despite these advances, existing systems typically combine multiple design choices, including prompting strategies, model selection, tool use, refinement, and execution-guided repair, making it difficult to isolate the contribution of explicit reasoning decomposition itself. In contrast, this work investigates reasoning decomposition as the primary architectural variable through a controlled comparison between reduced and full multi-stage pipelines under a consistent experimental setting, providing empirical evidence on how explicit reasoning decomposition influences SQL generation accuracy and robustness.

3 Proposed architecture

3.1 Overview

The proposed system decomposes NL2SQL into six sequential stages:

1. Question Analysis
2. Schema Selection
3. Query Planning
4. SQL Generation
5. SQL Refinement
6. SQL Validation

Data flow between agents is illustrated in Figure 1 (diagrammatic only; model assignment details: Table 2). The **4-step baseline** uses the same inputs (question + raw schema) but **drops the planning and refinement stages**, i.e.,

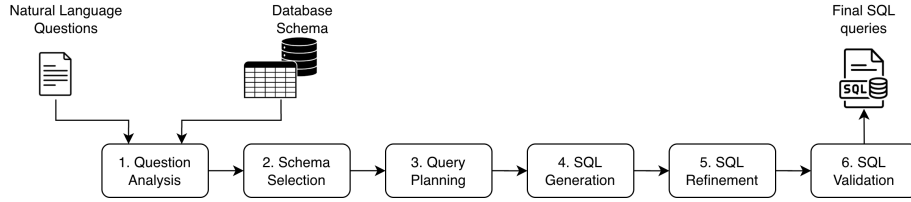


Fig. 1: Six-step pipeline (single forward pass). The 4-step baseline omits Query Planning and SQL Refinement.

it follows *Analysis* $\rightarrow$ *Schema* $\rightarrow$ *SQL Generation* $\rightarrow$ *Validation*. The **6-step pipeline** inserts *Query Planning* before SQL generation and *Refinement* immediately after the initial SQL, so the path from natural language to SQL remains sequential; the difference is that two intermediate reasoning stages are separated rather than collapsed into one generation pass.

Steps 5 and 6 (refinement vs. validation): In the reported implementation, orchestration is a **single forward pass** with no multi-round self-critique loop (Section 3.4). **Refinement** performs **one pass** of local semantic repair on the generated SQL, conditioned on analysis, filtered schema, and plan—it does **not** restart the cycle (e.g., it does not rerun the Planner or Generator) when an error is detected. **Validation** is a technical/syntax and schema-consistency check; the output is either **accepted SQL** or an **error report**—incorrect SQL is **not** treated as the final product, and the locked evaluation configuration does **not** trigger a full end-to-end pipeline retry.

Each agent handles a separate decision subproblem. The Question Analyzer extracts intent and expected output fields. The Schema Selector reduces schema noise. The Query Planner builds a logical plan before SQL is written. The SQL Generator produces initial SQL, the SQL Refiner performs one pass of semantic repair, and the SQL Validator checks technical consistency at the schema level and SQL executability.

The underlying hypothesis is simple: NL2SQL errors are easier to control when critical reasoning steps are made explicit rather than compressed into one generation pass.

3.2 Agent roles

Table 1 summarizes the roles of the six agents.

3.3 Design rationale

The architecture targets three common error groups: wrong output fields, weak logical planning, and unrepaired semantic mistakes after initial SQL generation. Accordingly, field prediction is isolated early, planning is separated from SQL writing, and refinement is limited to one pass. We compare a reduced 4-step baseline without the Planner and Refiner against the full 6-step pipeline to study

Table 1: Summary of the proposed six-step architecture.

Component	Input	Output	Main role
Question Analyzer	Question, raw schema	Intent analysis, expected output fields	Determines what the query should return
Schema Selector	Analysis, raw schema	Filtered schema	Reduces irrelevant tables and columns
Query Planner	Analysis, filtered schema	Logical execution plan	Organizes joins, filters, grouping, and set logic
SQL Generator	Analysis, filtered schema, plan	Initial SQL	Produces executable SQL
SQL Refiner	Initial SQL, analysis, schema, plan	Refined SQL	Repairs local semantic mistakes
SQL Validator	Refined SQL, filtered schema	Final SQL or error report	Checks syntax and schema consistency

whether these explicit checkpoints are associated with more robust behavior under a fixed protocol.

3.4 Implementation notes

The system uses fixed structured prompts in a modular multi-agent workflow. Each agent receives the natural language question, a schema representation, and relevant intermediate outputs from earlier stages. All experiments use deterministic decoding with temperature set to 0. At the implementation level, each agent prompt is fixed across four components: role description, structured input, reasoning/schema-consistency constraints, and output format. Orchestration is sequential in a single forward pass, with no voting, self-consistency, or multi-round self-critique loops.

For the full Spider 1.0 development-set results reported in this paper, the evaluated configuration is fixed as follows:

- Question Analyzer: GPT-4o
- Schema Selector: Gemini 2.5 Flash
- Query Planner: GPT-4o
- SQL Generator: GPT-4o
- SQL Refiner: GPT-4o
- SQL Validator: Gemini 2.5 Flash

This is the primary mixed-model variant used for full-development-set 6-step results in this paper. Internal benchmark runs with homogeneous model assignments (all Gemini 2.5 Flash, all GPT-4o, all Claude Sonnet 4, all Claude Opus 4) were less favorable on the accuracy–cost trade-off than the mixed configuration

above. The selected configuration reflects a pragmatic division of labor: GPT-4o covers structural reasoning and SQL generation–repair stages (Analyzer, Planner, Generator, Refiner), where internal benchmarks showed greater stability; Gemini 2.5 Flash handles schema filtering and final validation to balance cost and API-call latency.

4 Experimental setup

4.1 Dataset and metrics

We evaluate on Spider 1.0 [16], a standard cross-domain Text-to-SQL benchmark. Following current practice after the official Spider 1.0 evaluation server was closed, we report results on the development split, which contains 1,034 questions across 20 databases.

Difficulty (Easy / Medium / Hard / Extra) is assigned per question by the `eval_hardness` function in the Spider evaluation script (`experiments/test-suite-sql-eval/evaluation.py`, same protocol as [16]): difficulty is **inferred from the structure of the reference (gold) SQL** (counts of syntactic components such as joins, grouping, set operations, etc.), not from an external label we added outside SQL. Table 4 labels the hardest group **Extra Hard** to match common display conventions; in the script log the corresponding level is **extra**.

We use two standard metrics: Exact Match (EM), which measures structural equivalence to the reference SQL, and Execution Accuracy (EX), which measures whether the predicted query returns the same result as the gold query.

Statistical note: With `temperature = 0` and deterministic orchestration (Table 2), each configuration requires only **one** full evaluation pass over 1,034 questions; we do not report bootstrap confidence intervals because there is no LLM sampling randomness in this setup. (API variability or model-version drift remains a threat to reliability—see Section 6.)

4.2 Compared configurations

To clarify the value of each layer of reasoning decomposition, the paper reports four groups of configurations under the same evaluation protocol:

- baseline single-pass: direct prompting and chain-of-thought prompting
- baseline 4-step pipeline: Question Analysis → Schema Selection → SQL Generation → SQL Validation
- two 5-step ablations: dropping Query Planner or dropping SQL Refiner, respectively
- proposed 6-step pipeline: Question Analysis → Schema Selection → Query Planning → SQL Generation → SQL Refinement → SQL Validation

For the 6-step pipeline and the 5-step ablations in Table 3, per-agent model assignment matches Section 3.4 and Table 2 (GPT-4o for Question Analyzer, Query Planner, SQL Generator, and SQL Refiner; Gemini 2.5 Flash for Schema

Table 2: Implementation and reproducibility summary for the locked mixed-model 6-step configuration on the full development set (aligned with `agents.yaml` in the codebase). The 4-step baseline in Tables 3–4 uses a separate model assignment, as stated in Section 4.2.

Item	Value
Agent configuration (proposed 6-step)	GPT-4o for Question Analyzer, Query Planner, SQL Generator, and SQL Refiner; Gemini 2.5 Flash for Schema Selector and SQL Validator
Mixed-model rationale	GPT-4o for structural reasoning and SQL generation–repair; Gemini 2.5 Flash for schema filtering and final validation to balance cost and API latency in internal benchmarks
Temperature	0
Top-p / sampling	No additional tuning beyond temperature = 0; other sampling parameters fixed at provider API defaults and unchanged across compared configurations
Max output tokens	2048
Model access	Hosted API calls: Gemini 2.5 Flash, GPT-4o, and (for the 4-step baseline, Question Analyzer) Claude Sonnet 4; no fine-tuning and no self-hosted models
Schema representation	Structured Spider schema (tables, columns, keys); filtered sub-schema passed to downstream agents
Schema filtering policy	Keep tables and columns directly linked to entities, filter predicates, joins, and expected output fields extracted by the Analyzer
Prompting policy	One fixed template per agent; only the question, schema, and per-example intermediate outputs change
Orchestration details	Sequential single pass: each agent’s output is structured input to the next; no voting, self-consistency, or multi-round loops
Number of runs	One full evaluation pass over 1,034 questions per reported configuration; with temperature = 0, the pipeline runs in an application-level deterministic mode
Evaluation protocol	Official Spider evaluation script (EM / EX on SQLite)
Implementation stack	Modular multi-agent workflow (CrewAI-style orchestration) coordinating hosted API calls in a fixed agent order

Selector and SQL Validator). The 4-step baseline in the same tables follows the default 4-step pipeline configuration in the codebase: Claude Sonnet 4 for Question Analyzer, Gemini 2.5 Flash for Schema Selector and SQL Validator, GPT-4o for SQL Generator. The two single-prompt baselines in Table 3 use GPT-4o for the SQL generation call. Thus, the 4-step vs. 6-step comparison keeps the same evaluation protocol and dev split but does **not** fix the same backbone at the Question Analysis step; the discussion and limitations acknowledge this when interpreting architectural gaps.

Accordingly, Table 3 does not mix “planned” rows with “executed” rows; every configuration in the table is from a completed evaluation run. Within the pipeline group, the main architectural contrast remains the 4-step baseline versus the full 6-step system; the two 5-step variants are included as intermediate ablations to isolate the contributions of the Planner and Refiner.

4.3 Reproducibility information

Table 2 summarizes the locked evaluation configuration for the proposed 6-step pipeline on the full development set. Model assignment for the 4-step baseline and single-prompt baselines is stated separately in Section 4.2.

Table 3: Main internal comparison across prompting variants and pipeline configurations.

Configuration	EM (%)	EX (%)
Single-prompt system (direct SQL)	71.4	79.0
Single-prompt system (chain-of-thought)	72.6	80.0
4-step baseline	73.7	81.2
5-step without Planner	75.0	82.6
5-step without Refiner	76.1	83.9
6-step proposed	77.8	85.6

Table 4: Full Spider 1.0 development-set results by difficulty.

Difficulty	#Ex.	4-St. EX	4-St. EM	6-St. EX	6-St. EM
Easy	248	76.6	69.0	81.9	72.2
Medium	446	83.4	75.8	86.3	79.1
Hard	174	78.2	70.7	87.4	79.9
Extra Hard	166	85.5	78.3	87.3	80.1
All	1,034	81.2	73.7	85.6	77.8

5 Results and discussion

5.1 Main results

Table 3 presents the internal comparison structure used in this paper, spanning centralized prompting, reduced pipelines, and full six-step decomposition.

On the rows of Table 3, the proposed 6-step system improves over the reduced 4-step baseline by 4.1 EM points and 4.4 EX points. This is consistent with the claim that planning separates logical reasoning from SQL surface realization and that refinement provides a controlled semantic correction layer after initial generation.

At the same time, this evidence should be read conservatively. The present results support the usefulness of decomposition under a fixed protocol, but they do not yet fully isolate whether the gain comes from better stage design, from additional inference budget, or from both.

5.2 Results by difficulty

Table 4 breaks down full-development-set results by Spider difficulty (dataset-native labels). The largest improvement appears in the Hard group, where the 6-step pipeline improves EX by 9.2 points and EM by 9.2 points over the 4-step baseline.

The difficulty breakdown sharpens the main claim. Improvements are not limited to easy cases; they are strongest where query structure is more fragile, especially when joins, grouping, or set operations are involved. This pattern is

Table 5: Contextual comparison against representative Spider 1.0 systems across schema-aware parsing, prompt-based LLM methods, and decomposition-oriented methods. Values from prior work are shown only as literature context because reporting conditions differ.

System	EM (%)	EX (%)	Split / note
RAT-SQL + BERT [12]	65.6	—	Early schema-aware Spider milestone
BRIDGE + BERT (ensemble) [6]	71.1	—	Schema-aware parser with value grounding
PICARD + T5-3B [10]	70.6	75.7	Commonly cited Spider reference
RESDSQL + NatSQL [5]	76.7	78.2	Commonly cited Spider reference
DIN-SQL + Codex [9]	57.0	78.0	Prompt-based literature reference
DAIL-SQL + GPT-4 [2]	—	86.6	Spider leaderboard EX (in-context prompting)
SGU-SQL [17]	78.3	88.0	Reported Spider-dev (structure-guided)
MAC-SQL + GPT-4 [13]	—	86.8	Reported Spider-dev EX (multi-agent collaboration)
4-step baseline (this work)	73.7	81.2	Spider 1.0 dev, 1,034 questions
6-step proposed (this work)	77.8	85.6	Spider 1.0 dev, 1,034 questions

consistent with, but does not by itself prove, the value of explicit intermediate reasoning checkpoints.

5.3 Contextual comparison with reported Spider 1.0 results

One reviewer-facing question is where the proposed system sits relative to prior Spider-based work. To answer that directly, Table 5 broadens the comparison across three benchmark lineages on Spider 1.0: early schema-aware parsers (RAT-SQL, BRIDGE), later strong benchmark references (PICARD, RESDSQL), and recent LLM-era prompting or decomposition systems (DIN-SQL, DAIL-SQL, SGU-SQL, MAC-SQL). The comparison is contextual rather than strictly controlled because backbone models, prompting recipes, available database content, and sometimes evaluation splits or reporting conventions differ across papers. Some prior studies also report only one of the two metrics, so unavailable values are marked with an em dash.

The main takeaway is clearer when read by comparison group. First, relative to the schema-aware parsing line represented by RAT-SQL and BRIDGE, the proposed pipeline is substantially stronger, which suggests that explicit intermediate reasoning now matters at least as much as encoder-side schema representation. Second, relative to later benchmark references such as PICARD and RESDSQL, the proposed system remains competitive and exceeds their reported EX values in this contextual view. Third, relative to LLM prompting methods, the result is stronger than DIN-SQL, while DAIL-SQL still defines a stronger EX-only reference for highly optimized in-context prompting. Finally, compared

Table 6: Error taxonomy from non-exact-match predictions on full Spider 1.0 dev.

Error group	4-cnt	4-%	6-cnt	6-%	Interpretation
Subquery wrapper	47	17.3	47	20.4	Wrappers persist in both
LIMIT 0	37	13.6	27	11.7	Reduced in 6-step
Redundant CROSS JOIN	28	10.3	19	8.3	Planner/Refiner help
Predicate-flip risk	30	11.0	18	7.8	Reduced, not eliminated
Extra output column	12	4.4	4	1.7	Better in 6-step

with recent decomposition- and agent-oriented systems, the picture is more balanced: the 6-step pipeline remains one to two EX points below SGU-SQL and MAC-SQL, while relying on a strictly single-pass architecture without voting, iterative repair loops, or in-context example selection. In other words, the present system should be read not as a new leaderboard claim, but as evidence that a relatively simple six-stage control architecture can reach the same broad performance band as recent Spider-focused decomposition methods while staying interpretable through explicit intermediate checkpoints.

5.4 Log-based error pattern analysis

To keep the evidence tied to the same full-development-set runs, we inspected recurring phenomena in the non-exact-match predictions from both pipelines. The log contains **272** non-exact-match cases for the 4-step baseline and **230** for the 6-step system, consistent with the EM gap in Table 3. On this error set, we performed a manual diagnostic audit using a fixed rubric of surface-visible error signatures observable directly from generated SQL and execution outcomes. Labels in Table 6 are therefore descriptive evidence to support architectural interpretation, not an independent labeling scheme aimed at strong statistical inference. Counts are **not mutually exclusive** because a single prediction can exhibit multiple issues; percentages use each pipeline’s non-EM total as the denominator.

The main value of this analysis is controlled description rather than definitive causality. Not every phenomenon disappears, but the 6-step design reduces several high-value failure modes tied to output shape and unnecessary structural complexity. That behavior is consistent with the intended roles of the Planner and Refiner, although we do not treat this table as standalone proof beyond the main metrics.

5.5 Illustrative qualitative cases

Table 7 complements aggregate metrics with short illustrative qualitative examples. They are illustrative only, not a statistical sample.

Table 7: Illustrative qualitative cases (shortened).

ID	Question (short)	Pattern	One-sentence analysis
1	Movie titles with ratings both 3 and 4 stars	4-step uses OR; gold uses INTERSECT	Shorter pipelines can flatten intersect semantics
2	Airlines from a specific source airport	Predicate flip (= vs. !=)	Some errors persist despite decomposition
3	Documents not using a template	LIMIT 0 / wrappers	Residual errors in post-generation formatting

Table 8: Cost and latency profile by configuration.

System	Tokens/ex.	API calls	Rel. cost	p50 (s)	p90 (s)	EX (%)	Δ EX
Single-prompt direct	3450	1	1.00x	2.8	4.9	79.0	—
Single-prompt CoT	4080	1	1.18x	3.3	5.6	80.0	+1.0
4-step baseline	6180	4	1.79x	6.4	10.8	81.2	+2.2
5-step w/o Planner	7520	5	2.18x	8.1	13.5	82.6	+3.6
5-step w/o Refiner	8410	5	2.44x	9.1	15.1	83.9	+4.9
6-step proposed	9460	6	2.74x	10.7	17.9	85.6	+6.6

5.6 Practical implications

From an application perspective, the results support multi-agent LLM pipelines for natural language access to structured data. Although the evaluation is conducted on Spider rather than on an enterprise dataset, this architectural pattern matters for analytics settings where users need reliable access to relational data without writing SQL. The main practical implication is improved controllability for analytics and decision-support workflows rather than immediate production deployment readiness.

5.7 Inference cost and latency

Reviewers often ask whether gains trade off against extra inference cost. Table 8 summarizes the operational profile of the compared configurations.

6 Limitations and threats to validity

The paper intentionally limits itself to Spider 1.0 development-set evaluation. This is a practical choice because the official Spider 1.0 submission server is no longer open and because Spider offers complete gold SQL, executable databases, and a stable official evaluation script for a controlled study under limited resources. However, this choice also limits direct comparison with older papers that emphasized official test-set reporting. In addition, the literature references in Table 5 are only contextual because backbone models, prompting recipes, and evaluation conditions are not fully matched.

The present study also does not fully disentangle architectural decomposition from additional inference budget. The 6-step pipeline uses more stages than the 4-step baseline, so a stronger causal claim would require cost-matched ablations,

direct cost logging, and latency analysis from the same rerun package. The 4-step vs. 6-step comparison in Tables 3–4 is also coupled with a different backbone at the Question Analysis step (Claude Sonnet 4 vs. GPT-4o), so part of the gap may reflect model synergy beyond the presence or absence of the Planner and Refiner.

Priority follow-up experiment: run the 4-step baseline with the **same** GPT-4o at Question Analyzer (other steps unchanged from the baseline codebase) on the **same** 1,034 questions and protocol—to isolate the Planner/Refiner effect. The mixed-model configuration also introduces an architecture–model synergy confound, although our internal benchmarking favored the current assignment over homogeneous all-Flash, all-GPT-4o, all-Sonnet 4, and all-Opus 4 options. Additional diagnostic analysis by SELECT components (e.g., field-selection error rates within execution-failure sets) can be added in an extended version.

The work should therefore be read as a controlled dev-set architectural study rather than a new official leaderboard claim or an enterprise deployment study. Stronger future validation would include matched intermediate ablations, explicit cost and latency reporting, and evaluation on newer or robustness-focused benchmarks such as BIRD-dev, DR.Spider, Spider-DK, and Spider 2.0-lite.

7 Conclusion

This paper presented a six-step multi-agent architecture for NL2SQL that separates question understanding, schema reduction, logical planning, SQL generation, refinement, and validation. On the Spider 1.0 development set, the proposed 6-step pipeline achieved 77.8% Exact Match and 85.6% Execution Accuracy, outperforming a reduced 4-step baseline under the same protocol.

The 6-step results correspond to the locked mixed-model variant (Table 2).

Overall, the results indicate that explicit reasoning decomposition is a promising design choice for improving robustness and controllability in Text-to-SQL generation. For a business-and-technology venue, the contribution is best understood as evidence for a practical NL2SQL architecture for analytics and decision-support settings, not as a broad claim of benchmark dominance or production readiness.

Bibliography

- [1] Harrison Chase. Langchain. <https://python.langchain.com>, 2024. Accessed 2024–2026.
- [2] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145, 2024.
- [3] Zhibin Gou, Zhihong Shao, Yeyun Gong, et al. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *Proceedings of ICLR*, 2024.
- [4] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, Jayant Krishnamurthy, and Luke Zettlemoyer. Learning a neural semantic parser from user feedback. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 963–973, 2017.
- [5] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 13067–13075, 2023.
- [6] Xi Victoria Lin, Richard Socher, and Caiming Xiong. Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing. In *Findings of EMNLP*, 2020.
- [7] João Moura et al. CrewAI: Open-source framework for multi-agent collaboration. <https://github.com/crewAIInc/crewAI>, 2024. Accessed 2024–2026.
- [8] Yafeng Nan, Haifeng Sun, Zirui Zhuang, Qi Qi, Guojun Chu, Jianxin Liao, Dan Pei, and Jingyu Wang. DIVER: A robust text-to-SQL system with dynamic interactive value linking and evidence reasoning. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD)*, 2026.
- [9] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of EMNLP*, 2021.
- [11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In *Proceedings of ACL*, 2020.
- [13] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li.

- MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pages 540–557, 2025.
- [14] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling (COLM)*, 2024.
 - [15] Tao Yu, Michihiro Yasunaga, Kai Yang, Rui Zhang, Dongxu Wang, Zifan Li, and Dragomir Radev. SyntaxSQLnet: Syntax tree networks for complex and cross-domain text-to-SQL task. In *Proceedings of EMNLP*, 2018.
 - [16] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of EMNLP*, 2018.
 - [17] Qinggang Zhang, Hao Chen, Junnan Dong, Shengyuan Chen, Feiran Huang, and Xiao Huang. Structure-guided large language models for text-to-SQL generation. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR 267, pages 74671–74691, 2025.